

OVERVIEW

Personal Portfolio is an interactive site showcasing data science and engineering projects. It is built with React, TypeScript, and Vite, and features 3D animations and a chat assistant.

Key features

- Custom hash routing
- Animated 3D folders
- Circular navigation lightbox
- Force-directed conceptual graph
- Interactive CV
- Self-hosted bilingual blog

TECH STACK

Core

React 18 on TypeScript 5.6 with strict mode, bundled with Vite 5.4.

Styles and animations

Tailwind CSS, lottie-react for the hero animation, CSS 3D transforms for the folders, custom SVG force-directed simulation for the concept graph.

Browser APIs

Web Audio API with GainNode, localStorage, Pointer Events for drag and pinch zoom, Page Visibility to pause music, MediaQuery to detect hover and theme preference.

Infrastructure

pnpm as the required package manager, GitHub Actions for CI, GitHub Pages for global static hosting with HTTPS.

Final bundle

Approximately 180 kilobytes gzipped. No React Router, Framer Motion or state management libraries.

FRONTEND ARCHITECTURE

The application is structured in five vertical layers that separate concerns: development tooling, main framework, styles and UI, browser APIs, and continuous deployment. Each layer only depends on the layers above and exposes a narrow interface to those below.

Vertical layers

Dev/Build (pnpm, TypeScript, Vite) which only depend on system tools. Core (React, Hash Routing) which mounts the site. Styles and UI (Tailwind, lucide, lottie, CSS 3D) which provide the visual identity. Browser APIs (Web Audio, localStorage, Pointer Events, Page Visibility, MediaQuery) which leverage native capabilities without extra libs. CI/Deploy (GitHub Actions and Pages) which ships the compiled bundle to production.

Key decision

A custom hash routing of about thirty lines replaces react-router, removing a significant dependency while keeping shareable URLs that survive browser refreshes. See the full stack diagram in the portfolio's Diagrams section.

MAIN COMPONENTS

FolderPortfolio

Renders the grid of six 3D folders with a fan of cards and orchestrates the modal lightbox.

ProjectDetailView

Detail view with hero, force-directed concept graph and six section cards. Navigate between projects without going back to the grid.

ConceptGraph

Custom physics simulation with node repulsion, edge springs and continuous levitation. Supports drag with pointer events and group filtering.

DiagramsViewer

Fullscreen viewer with zoom up to eight hundred percent, pan and pinch zoom on mobile. Circular navigation between multiple images.

PortfolioChat

Floating bubble with a collapsible panel. Mock by keyword matching, LLM integration planned for the next phase.

BACKEND ARCHITECTURE

A minimal serverless layer that coexists with the static frontend without turning it into a monolith. Two runtime workers for chat and administration, a suite of batch agents in GitHub Actions that auto-generate portfolio content.

Three functional layers

Events (push to repo, release tag, cron, visitor chat, admin approval) trigger the chain. Backend (runtime workers for chat and draft admin, batch agents in GitHub Actions grouped into content, lifecycle and distribution, mixed persistence between committed JSON and ephemeral Workers KV). Outputs (auto-deployed portfolio web, auto-generated CV PDF, blog on Dev.to, post on LinkedIn, chat response).

Key decision

Hybrid JAMstack instead of an Express or n8n monolith. The unpredictable traffic of a personal portfolio justifies pay-per-request on Cloudflare Workers rather than maintaining a 24/7 server. See the full flow diagram in the portfolio's Diagrams section.

DATA MODEL

The catalog of projects lives in `src/data/projects.ts` as an array of six categories. Each project declares a unique identifier, image, bilingual title and description, list of tags, a status from three values, links to repository and demo, graph concepts and optional cross-discipline categories.

Key types

LocalizedString, ProjectStatus, ConceptGroup with four semantic values: architecture, data, operations and machine learning.

Persistence

JSON and markdown committed to the repository for public data. Workers KV for chat rate limiting and the queue of distribution drafts. Conversation history is not stored, for privacy.

NEXT STEPS AND RECENT CHANGES

Recent changes

Recent changes include adding support for external repositories in agents, updating documentation, and improving chat assistant functionality. A self-hosted bilingual blog has also been added.

Next steps

- Implement new projects
- Improve user experience
- Add features to the chat assistant
- Optimize site performance